SMARTSTORE NAVIGATOR

A DSA + OOP project in C++, visualized through a full-stack web app

Dijkstra route planning · custom Queue & Priority Queue · Graph & Hash Map · OOP class hierarchy

C++17 **React + Vite** **Node.js / Express** **MongoDB Atlas** **Socket.IO**

SmartStore Navigator – Complete Workflow

C++ DSA Engine + Full Stack Visualization

1. USER

Uses the web application

2. FRONTEND (React + Vite)

User Interface (Login, Inventory, Route Planner, etc.)

HTTP Requests (Axios)

JSON Response

3. BACKEND (Node.js + Express)

REST API
Authentication (JWT)
Business Logic

Read / Write (CRUD)

Data

4. DATABASE (MongoDB Atlas)

Stores Users, Products, Inventory, etc.

Invoke C++ Engine

JSON Result

5. C++ DSA ENGINE (route_engine)

- Graph (Store Layout)
- Dijkstra's Algorithm
- BFS
- Binary Search
- Priority Queue (Min Heap)
- Queue Manager

Computes Optimal Route & Best Checkout Counter

JSON Response (Route / Queue Result)

HOW IT WORKS (STEP-BY-STEP)

1. **User Login / Signup** — User enters credentials. Backend verifies with MongoDB and returns a JWT token.
2. **Manage Inventory** — Add / Update / Delete products with aisle numbers in the store. Data is saved in MongoDB.
3. **Select Items (Aisle Numbers)** — User selects products to buy. Frontend sends the list of aisle numbers to the backend.
4. **Route Planning (DSA in C++)** — Backend calls the C++ engine which builds the graph and runs Dijkstra's Algorithm to find the shortest path visiting all required aisles.
5. **Optimal Route Returned** — C++ engine returns the optimal path and total distance. Backend sends the result to the frontend.
6. **Queue Recommendation (DSA in C++)** — Backend sends current queue data to C++ engine. Priority Queue (Min Heap) finds the counter with minimum waiting time. Recommendation returned to the frontend.
7. **Dashboard Updates in Real-Time** — Frontend displays optimal route, recommended counter, inventory stats, and other analytics. Socket.IO can be used for real-time updates.

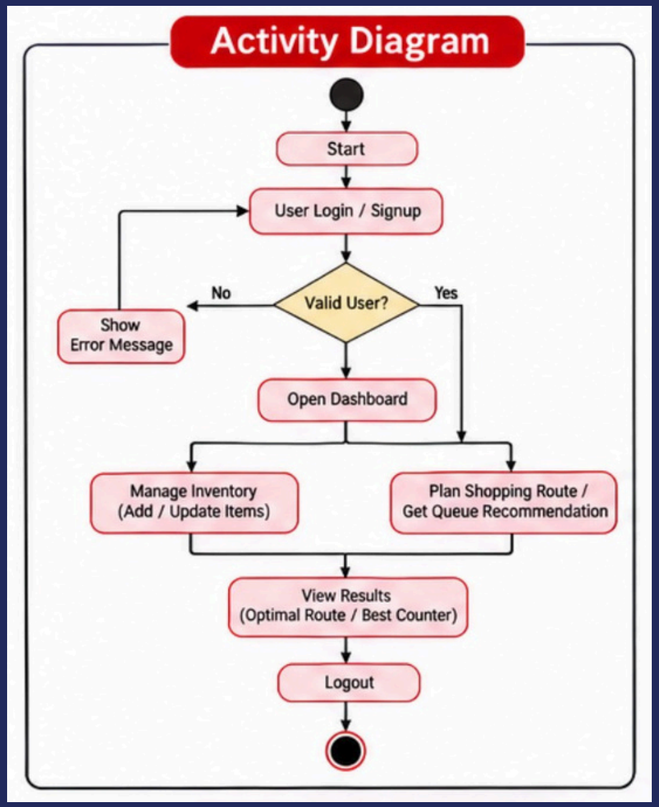
TECHNOLOGIES USED

React, Node.js, Express, MongoDB, C++, Socket.IO

DSA & OOP CONCEPTS USED

- ✓ Graph
- ✓ BFS
- ✓ Dijkstra's Algorithm
- ✓ Binary Search
- ✓ Queue
- ✓ Priority Queue
- ✓ Unordered Map
- ✓ OOP: Inheritance
- ✓ Abstraction
- ✓ Polymorphism
- ✓ Encapsulation

The SmartStore Navigator is a full-stack web application powered by a high-performance C++ Data Structures & Algorithms (DSA) engine designed to optimize the in-store shopping experience.



System Architecture

The C++ engine does the real computation — the web stack only displays it

1. **React (Vite) frontend**

   Sends requests, renders the route & inventory visually

2. **Express + Socket.IO backend**

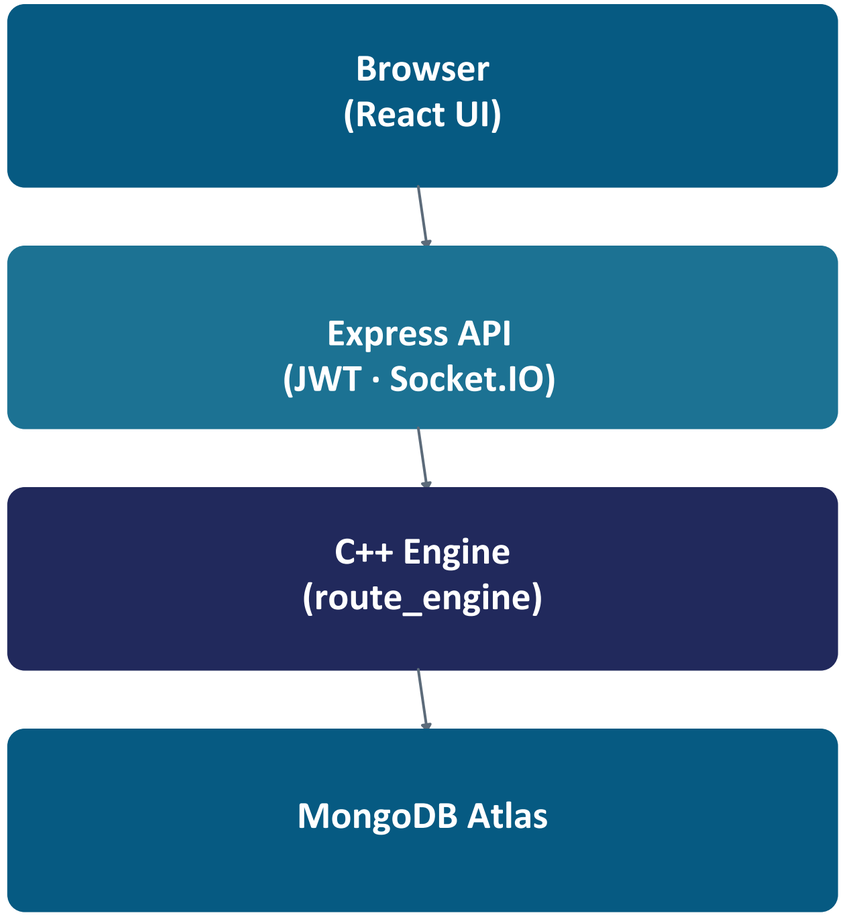
   Handles auth (JWT), MongoDB data, live updates

3. **C++ engine (route_engine)**

   Spawned as a child process — runs Dijkstra, BFS, Priority Queue

4. **MongoDB Atlas**

   Persists users and product inventory



MongoDB is queried directly by Express; the C++ engine only computes routes & queues

DSA Concepts — Implemented in C++

Every data structure below is hand-built, not just std:: called blindly

Graph

Adjacency-list store layout; aisles are nodes, walking distances are weighted edges

`Graph.h / .cpp`

Dijkstra's Algorithm

Computes the shortest multi-stop shopping route from the entrance

`RoutePlanner.cpp`

Queue (custom)

Array-backed FIFO queue, built from scratch — powers graph BFS traversal

`QueueManager.h`

Priority Queue

Manual binary min-heap ranks checkout counters by estimated wait time

`QueueManager.h`

Hash Map

unordered_map gives O(1) average product lookup inside the inventory

`InventoryManager.cpp`

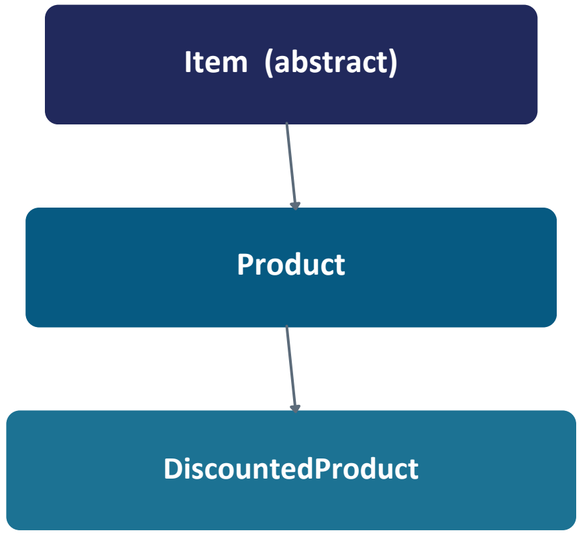
Binary Search

O(log n) lookup of an aisle id within a sorted aisle list

`RoutePlanner::binarySearchAisle`

OOP Design — Class Structure

All five pillars of OOP are demonstrated across the C++ class hierarchy



RoutePlanner

has-a Graph

QueueManager

has-a PriorityQueue

1 **Encapsulation**

Private fields with getters/setters (Product, InventoryManager)

2 **Abstraction**

Item exposes only display() / getPrice() as pure virtual methods

3 **Inheritance**

Product : Item, DiscountedProduct : Product

4 **Polymorphism**

getPrice() and display() overridden and called via base Item*

5 **Composition**

RoutePlanner has-a Graph; QueueManager has-a PriorityQueue

Features at a Glance

What the app demonstrates — powered by the C++ engine underneath

Route Planner

Pick products, get the Dijkstra-shortest walking route

Inventory

Add / edit / delete / search products, stored in MongoDB

Queue Manager

Priority-queue recommends the fastest checkout counter

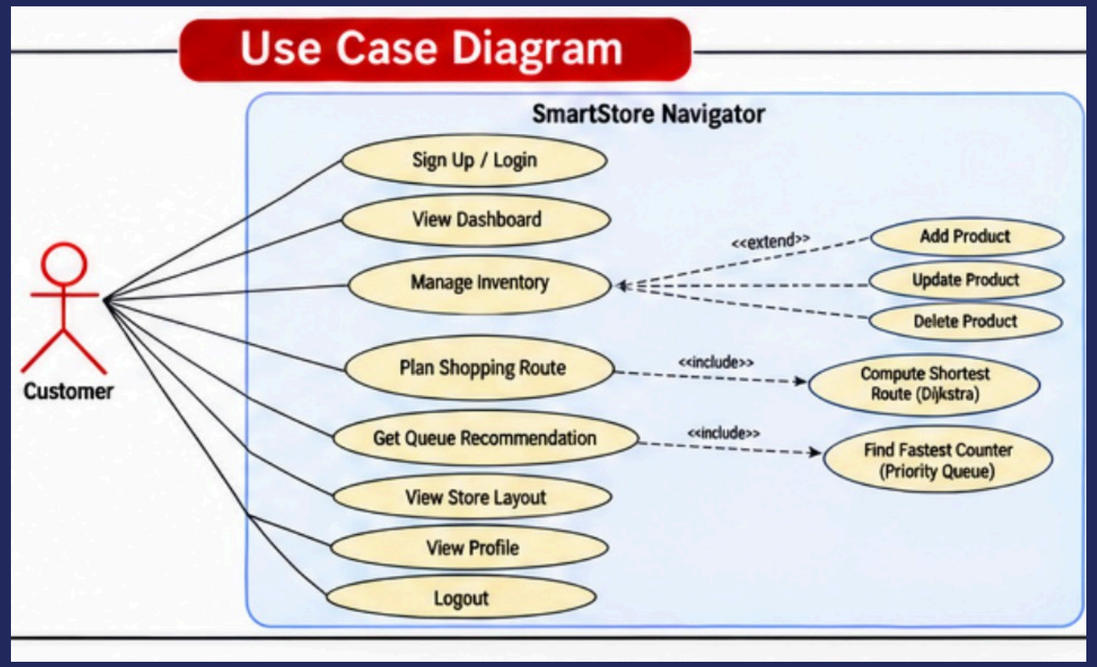
Dashboard

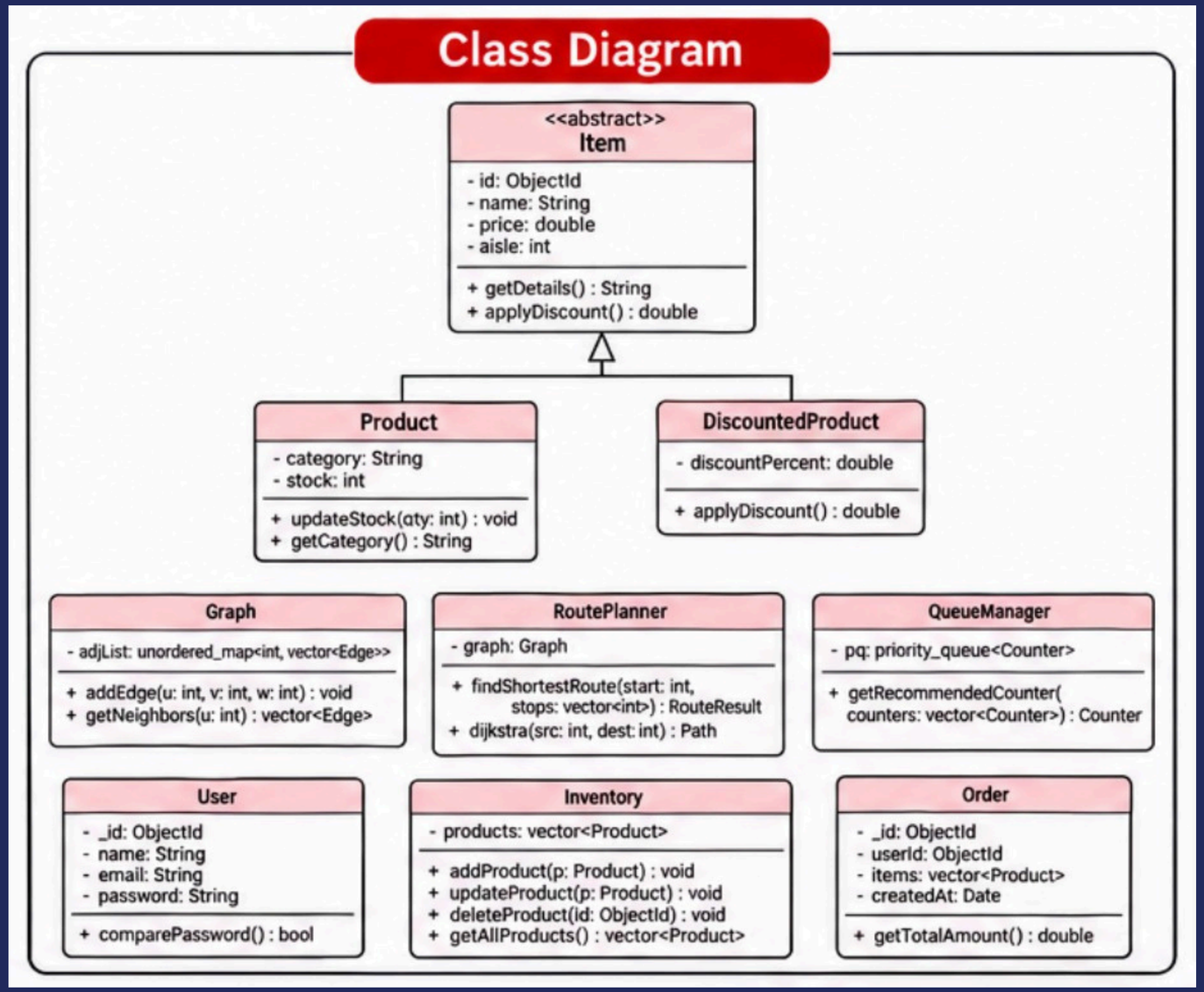
Live stats via Socket.IO: products, customers, route, queue

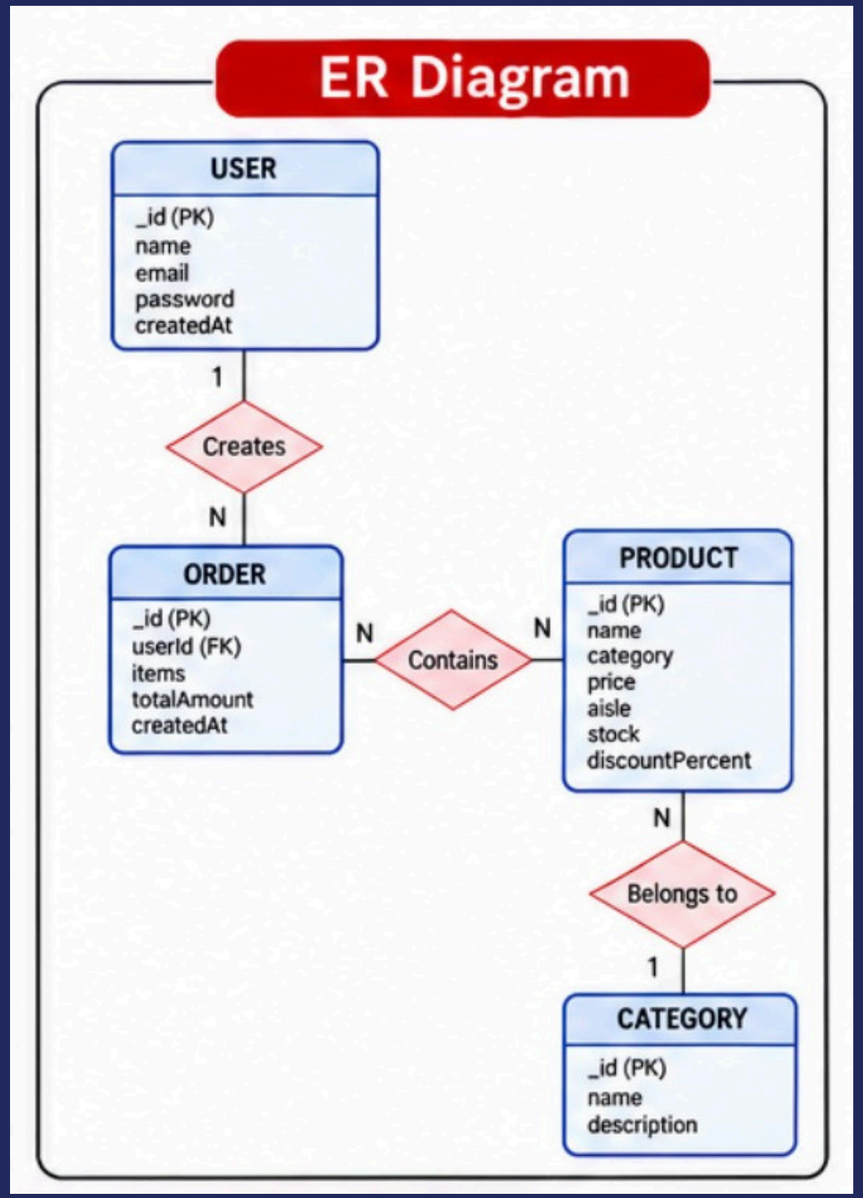
It provides the UML architecture for the SmartStore Navigator application through a Use Case Diagram and a Class Diagram.

It details how the Customer interacts with the system:

- Account Management: Sign Up / Login, View Profile, and Logout.
- Dashboard & Navigation: View Dashboard, View Store Layout, and Manage Inventory (Add, Update, Delete Product).
- Smart Recommendations:
  - Plan Shopping Route (includes computing the shortest path via Dijkstra's Algorithm).
  - Get Queue Recommendation (includes finding the fastest checkout counter using a Priority Queue).







the execution flow when a user requests an optimal shopping route:

Request: The user selects products on the React Frontend (Vite) and submits a request.

API Call: The frontend sends a POST /api/route request containing the chosen item stops to the Express Backend (Node.js).

Bridge Execution: The backend calls routeEngine.js, which spawns an external process executing the C++ Engine (route_engine.exe).

Algorithmic Computation: The C++ engine executes Dijkstra's Algorithm using store graph data to find the shortest shopping path and distance.

Response Flow: The calculated path/distance is returned as a JSON result through routeEngine.js back to the backend, which forwards it to the frontend to display the final optimal route to the user.